



CLINICAL BIOMARKER RESEARCH LABORATORY

CBRL Website

Content Dashboard & System Documentation

Version	2.0.0
Application	Next.js 16 · React 19 · TypeScript 5
Live site	cbrl.iitkgp.ac.in
Institution	School of Medical Science and Technology, IIT Kharagpur
Audience	Content editors and system administrators

This document has two parts. **Part I** is written for the people who edit the website day to day and assumes no technical background. **Part II** is written for whoever installs, configures, secures and maintains the server, and assumes command-line familiarity.

Contents

Part I — Content Editor's Guide

1	What the dashboard controls
2	Signing in
3	Sessions and the inactivity timeout
4	How every editor screen works
4.1	The sidebar and the save bar
4.2	Adding, reordering and removing entries
4.3	Saving, discarding and validation errors
4.4	Uploading images
5	Section by section
5.1	Overview
5.2	Members
5.3	Publications
5.4	Projects
5.5	Research areas
5.6	Gallery
5.7	Facilities
5.8	Citation metrics
6	Field limits and formats
7	When something goes wrong

Part II — System Administrator's Guide

8	Architecture
9	Installation and the build
10	Environment variables
11	How content is stored and revalidated
12	Authentication and session model
12.1	The session cookie
12.2	Absolute expiry and idle timeout

12.3 The activity heartbeat

13 Security layers

14 Reverse proxy configuration

15 Deployment and keeping git in step

16 Deploying elsewhere (Vercel and other hosts)

17 API reference

18 Backup and restore

19 Routine maintenance

20 Administrator troubleshooting

PART I

Content Editor's Guide

Everything you need in order to keep the CBRL website up to date — adding a new lab member, recording a publication, publishing photographs from a conference — without touching any code.

Who this part is for

Anyone given the dashboard password: research scholars, staff, or the PI. No programming knowledge is assumed. You will need a web browser and the address of the site.

1 What the dashboard controls

The dashboard lives at /admin on the website — for the live site that is <https://cbrl.iitkgp.ac.in/admin>. It edits the parts of the site that change often, and it publishes instantly: there is no separate "deploy" step and no waiting for a developer.

Dashboard section	Public page	What you can change
Overview	—	A summary of what is currently on the site. Nothing is edited here.
Members	/members	Faculty, postdoctoral fellows, research scholars, staff and alumni — including photographs and profile links.
Publications	/publications	The list of peer-reviewed papers. Add by DOI, paste BibTeX, or fill in a form.
Projects	/projects	Ongoing funded projects, their objectives, the completed-projects PDF, and sponsor logos.
Research areas	/research	The research areas and the disease topics written up under each.
Gallery	/gallery	Photo albums, their categories, cover images and captions.
Facilities	/facilities	Instruments, their features and in-charges, headline figures, and slot charges.
Citation metrics	/publications	Total citations, h-index, and the source link shown alongside the publication list.

EVERYTHING ELSE

Page wording outside these sections — the homepage introduction, the contact details in the footer, the About the PI biography — is part of the site's code and needs a developer. If you are unsure whether something is editable, look for it in the sidebar; if it is not there, it is not editable from the dashboard.

2 Signing in

- 1 Go to <https://cbrl.iitkgp.ac.in/admin> in any modern browser.
- 2 Enter the dashboard password. There is one shared password for everybody — there are no individual user accounts.
- 3 You arrive at the Overview screen. If you were trying to reach a particular section before signing in, you are returned to it automatically.

If the password is refused

The message tells you how many attempts remain. After **five** wrong attempts your network address is blocked for **ten minutes**; waiting is the only way through, so find the correct password rather than guessing again.

If you see "The dashboard is not configured yet"

The server has not been given a password. This is not something you can fix from the browser — contact your system administrator and point them at section 10 of this document.

SHARING THE PASSWORD

Because there is a single shared password and no per-person accounts, the site cannot show who made a given change. Treat the password as you would a key to the lab: share it only with people who need to edit the site, and ask your administrator to change it whenever somebody with access leaves the group.

3 Sessions and the inactivity timeout

Being signed in does not last indefinitely. Two separate limits apply, and whichever is reached first signs you out.

Limit	Duration	Behaviour
Inactivity timeout	5 minutes	Five minutes with no typing, clicking, scrolling or mouse movement ends the session. Any activity resets the clock.
Absolute session length	8 hours	Eight hours after signing in the session ends regardless of how busy you have been. Sign in again to continue.

3.1 The warning

One minute before the inactivity timeout expires, a panel appears in the middle of the screen titled "**Still there?**" with a countdown. You have three choices:

- **Stay signed in** — dismisses the warning and resets the five minutes.
- **Sign out** — ends the session immediately.
- **Do nothing** — the countdown reaches zero and you are returned to the sign-in screen, which explains that you were signed out for inactivity.

Simply moving the mouse or pressing a key also dismisses the warning, because that counts as activity.

UNSAVED WORK IS LOST

Being signed out discards anything you have typed but not saved. The warning exists to give you a minute to press **Save changes** first — treat it as a prompt to save, not as a notice you can ignore. If you are interrupted mid-edit, save before you walk away.

3.2 Why five minutes, and what counts as activity

The timeout is deliberately short because the dashboard can change the public website. Typing counts as activity even though nothing is being sent to the server while you write, so composing a long paragraph will *not* sign you out. Activity in one browser tab keeps every other dashboard tab signed in as well.

If five minutes proves too disruptive for the way your group works, the limit is a server setting and your administrator can raise it — see section 10.

4 How every editor screen works

All seven editable sections share the same layout and the same controls. Learn them once and every screen behaves the same way.

4.1 The sidebar and the save bar

The **sidebar** on the left lists every section. On a narrow screen or a phone it is hidden behind the menu button at the top left.

The **save bar** is pinned to the bottom of the screen and is the only place work is committed. It always tells you the current state:

What it says	What it means
No changes yet	Nothing has been touched since the page loaded.
Unsaved changes	You have edited something. It exists only in your browser.
Saving...	The change is being written on the server.
Saved — the live page is updated	Done. The public page already reflects it.
Last saved 5 minutes ago	Everything is saved; this is when it happened.

The **View page** button at the top right of each section opens the public page it drives in a new tab, so you can check a change immediately after saving.

4.2 Adding, reordering and removing entries

Lists — members, projects, albums, instruments — are shown as collapsed cards, numbered, with the title on the left. Click a card to expand it and edit its fields; click again to collapse.

Control	Effect
Add button	Appends a new blank entry at the end of the list. Expand it and fill it in.
▲ / ▼ arrows	Move the entry one place up or down. This order is exactly the order visitors see. The arrows grey out at the ends of the list.
Bin icon	Removes the entry — click once to arm it, and the row turns red asking you to confirm; click the bin a second time to remove it. Clicking anywhere else cancels.

NOTHING IS PERMANENT UNTIL YOU SAVE

Adding, reordering and removing all happen in your browser only. Nothing reaches the website until you press **Save changes** — and until then, **Discard changes** puts everything back exactly as it was.

4.3 Saving, discarding and validation errors

Press **Save changes** to publish. The server checks the whole document before writing anything. If something is wrong, *nothing* is saved and the save bar lists the problems, each naming the exact field, for example:

```
students[4].email must be a valid email address
albums[2].image   is required
```

Read this as: the fifth research scholar has a malformed email address, and the third gallery album has no cover image. Counting starts at zero, so `[4]` is the fifth entry. Fix the named fields and save again. At most twelve problems are listed at once; if there are more, the list ends with a count of the remainder.

Discard changes throws away everything unsaved and reloads what is currently published. It cannot be undone, but it also cannot affect the live site.

WHAT THE SERVER TIDIES UP FOR YOU

Saving normalises the document: spaces at the beginning and end of fields are trimmed, blank rows in lists are dropped, and duplicated identifiers are made unique. The form redraws to show exactly what was stored, so if a field looks slightly different after saving, that is why.

4.4 Uploading images

Anywhere the site shows a photograph you get an image box: a small preview, the current image path, and an **Upload image** button (**Replace image** once one is set).

- 1 Click **Upload image** and choose a file. JPEG, PNG, WebP, AVIF and GIF are accepted, up to **12 MB**.
- 2 The server converts it to WebP, rotates it upright, and resizes it to a sensible width for where it will appear. The path fills in automatically.
- 3 Press **Save changes**. The upload alone does not publish anything.

Clear removes the image from the entry. It does not delete the uploaded file from the server.

WHY REPLACED PHOTOS APPEAR INSTANTLY

Every upload is given a filename containing a fingerprint of its contents, so a replacement always arrives at a brand-new address. That is why you never have to clear your browser cache to see an updated photograph.

You may also type a path directly into the box — useful for an image already on the server. It must begin with `/` and point inside the site, for example `/images/MembersDP/koe1-chaudhury-a1b2c3d4.webp`. Full web addresses beginning `http://` or `https://` are rejected.

5 Section by section

5.1 Overview

The landing screen after signing in. It reports how many members, publications, projects, albums and instruments are currently published, and links to each section. Nothing is editable here — it is a place to confirm the site holds what you expect.

5.2 Members

Members are divided into five groups, each shown as its own tab or block, and each rendered as a separate band on the public page:

Group	Shown on the site as
faculty	Faculty
postdocs	Postdoctoral Fellows
students	Research Scholars
staff	Staff
alumni	Alumni

Fields

Field	Required	Notes
Name	Yes	Up to 160 characters.
Title	Yes	Designation, e.g. "Senior Research Scholar". Up to 160 characters.
Photo	No	Uploaded to the members or alumni folder. A missing photo falls back to a placeholder.
Email	No	Two addresses may be given, separated by a comma.
Phone	No	Up to 60 characters.
Bio	No	Up to 2,000 characters.
Research	No	One-line research interest. Up to 500 characters.
Qualifications	No	Up to 2,000 characters.
Joined	No	Must be written <code>YYYY-MM</code> — for example <code>2023-08</code> . Any other format is rejected.
Responsibilities	No	Mainly used for staff. Up to 2,000 characters.
Current position	No	Alumni only — where they are now. Up to 400 characters.
Thesis title	No	Alumni only. Up to 600 characters.
Profile links	No	Google Scholar, ORCID, LinkedIn and ResearchGate. Each must start with <code>https://</code> .

RENAMING A MEMBER BREAKS THEIR LINK

Each member has a hidden identifier derived from their name the first time they are saved, and the public site links to individual cards using it — `/members#ankush-das`. Changing a spelling is fine, but be aware that any link shared earlier to that specific card may stop working. If two members would end up with the same identifier, the second is automatically given a numbered suffix.

Moving somebody to Alumni

There is no automatic transfer. Remove the person from their current group, add them under Alumni, and re-enter their details — adding **Current position** and **Thesis title**, which only Alumni displays. Do it in a single sitting and save once.

5.3 Publications

The publications page is generated from a BibTeX record of roughly 180 entries. The editor works on **one entry at a time** — every other entry in the file is left untouched, byte for byte — so there is no risk of a stray edit disturbing the rest of the list.

Three ways to add a paper

Method	How it works
By DOI <i>recommended</i>	Paste the DOI — <code>10.1016/j.bios.2026.118980</code> , or the full <code>https://doi.org/...</code> address — and the server fetches the full record from doi.org. Check the filled-in fields and save.
Paste BibTeX	Paste an entry copied from the publisher or Google Scholar. It must start with <code>@article{</code> or similar. Several entries can be pasted at once and they keep their order.
Fill the form	Type the title, authors, journal, year and so on by hand. Use this when a paper has no DOI yet.

New entries are added at the **top** of the record. The public page groups everything by year regardless, so position within the file does not affect what visitors see.

Editing and deleting

Existing entries are listed with their title, authors and year. Open one to edit its fields, or delete it outright. Fields the public page does not display — `keywords`, `issn`, `abstract` — are preserved even though the form does not show them.

DUPLICATE CITATION KEYS

Every entry carries a citation key, which must be unique. If a paste or a DOI import produces a key already in use, the entry is flagged as a duplicate. Give one of them a slightly different key — appending `a` or the journal abbreviation is the usual fix.

If a DOI lookup fails

Message	What to do
"That does not look like a DOI"	The DOI must start <code>10.</code> followed by four or more digits, a slash, then the suffix. Remove any surrounding text.
"No record found for that DOI"	The DOI is not registered. Check it against the publisher's page; very new papers sometimes take days to appear.
"Could not reach doi.org"	The server has no internet access, or doi.org is down. Paste the BibTeX instead — that works entirely offline.

5.4 Projects

Three separate things live here.

Ongoing projects

Each card carries a title and a short description (both required), plus duration, funding agency, your role, a detailed description, methodology and expected outcomes. **Objectives** is a list — add one line per objective; blank lines are dropped when you save. The **icon** is a single emoji shown beside the title and defaults to 🧑🔬 if left empty.

Completed projects PDF

A single link to the PDF listing finished projects. It must be a path inside the site beginning `/`, such as `/data/completed-projects.pdf`. Replacing the document is a job for your administrator — the dashboard

uploads images only, not PDFs.

Sponsor logos

Each sponsor needs an image, descriptive alt text for screen readers, and its **width and height in pixels**. All four are required. The dimensions stop the page jumping about while logos load, so they must match the real image — your image viewer or the file's properties will tell you.

5.5 Research areas

A two-level structure. Each **area** has a number badge, a title, a tagline and a description. Inside it sit **topics**, shown on the public page as an expanding accordion.

Each topic has a title, a one-line summary, and a **body** made of separate paragraphs — add one box per paragraph rather than pressing Enter inside a single box, which is what produces the spacing on the published page.

The index badge (**01**, **02**, ...) is filled in automatically from the position of the area if you leave it blank, so reordering areas rennumbers them.

5.6 Gallery

Categories are defined first, as a simple list — "Conferences", "Awards", "Lab outings". Every album must then be given a category *from that list*; a category that is not in the list is rejected when you save.

Each album needs a title and a **cover image** (both required), and may also carry a caption, a date, a longer description and any number of additional photographs. Gallery uploads may be filed into a sub-folder, which keeps large albums tidy on the server.

REMOVING A CATEGORY

Deleting a category from the list while albums still use it makes those albums fail to save, with an error naming the allowed categories. Move the albums to another category first, then remove the old one.

5.7 Facilities

Three groups of content:

- **Headline figures** — the short statistics across the top of the page. Each is a value (up to 20 characters, e.g. "12+") and a label (up to 40 characters, e.g. "Major instruments"). Both required.
- **Instruments** — title, photograph and its alt text are all required. Each may also list features, a funding source, and an in-charge with name, title, email and a profile link.
- **Slot charges** — each charge block has a title, a duration, and a list of price rows (label and value, both required, such as "IIT KGP internal" / "₹500"). The note below the table is edited in **Charges note**.

5.8 Citation metrics

The small panel of figures on the publications page. Four fields, all required:

Field	Notes
Total citations	Free text, so "4,250" and "4250+" are both fine. Up to 20 characters.
h-index	A whole number between 0 and 500.
Source name	Usually "Google Scholar".
Source URL	Must start <code>https://</code> .
Last updated	Must be written <code>YYYY-MM-DD</code> — for example <code>2026-07-27</code> .

These figures are typed in by hand; nothing fetches them automatically. Refreshing them once a term is a sensible habit.

6 Field limits and formats

A reference for the formats the server insists on. Getting one of these wrong is the most common reason a save is rejected.

Rule	Applies to
Dates as YYYY-MM	Member "Joined" — 2023-08 , not Aug 2023 .
Dates as YYYY-MM-DD	Citation metrics "Last updated" — 2026-07-27 .
Links	Must start with https:// , http:// , mailto: or / . A bare www.example.com is rejected.
Image and file paths	Must start with a single / and stay inside the site. Full web addresses and .. are rejected.
Email addresses	Must contain @ and a dot in the domain. Two addresses may be separated by a comma.
Uploads	JPEG, PNG, WebP, AVIF or GIF, maximum 12 MB. Everything becomes WebP.
Blank rows	Silently dropped from lists such as objectives, features and topic paragraphs.
Leading/trailing spaces	Trimmed from every text field on save.

7 When something goes wrong

What you see	What it means and what to do
"Some fields need attention before this can be saved."	Validation failed and <i>nothing</i> was written. Each listed line names the field. Fix them and save again — see section 4.3.
"Could not write the file on the server. Check file permissions."	The server could not save to disk. Nothing was lost from your screen — do not reload. Copy your work somewhere safe and contact your administrator; this is section 20 for them.
"Could not reach the server."	Your network connection dropped, or the site is down. Your edits are still on screen. Restore the connection and press Save again.
Sent back to the sign-in page unexpectedly	The session expired — five minutes idle or eight hours total. Sign in again. Unsaved work is gone; see section 3.
"Signed out after a period of inactivity."	Confirmation of the five-minute timeout. Sign in and continue.
"Too many attempts. Try again in N minutes."	Five wrong passwords. Wait it out — there is no way to shorten it from the browser.
"The dashboard is not available from your network."	An address restriction is in force and yours is not on the list. Try from the campus network, or ask your administrator — section 13.
"This request did not come from the dashboard."	Usually a stale browser tab left open a long time. Reload the page and try again.
"That image is larger than 12 MB."	Resize or compress the photograph before uploading. Anything much above 2,000 pixels wide is larger than the site needs.
"That file could not be read as an image."	The file is corrupt, or it is not really an image — a PDF renamed <code>.jpg</code> , for instance. Re-export it and try again.
A saved change is not visible on the public page	Confirm the save bar said "Saved". Then reload the public page with <code>Ctrl + F5</code> . If it is still missing, tell your administrator which section and field.

REPORTING A PROBLEM USEFULLY

Include the section you were editing, the exact wording of the message, roughly when it happened, and whether you had just uploaded an image. The server keeps logs, and a timestamp makes the matching entry easy to find.

PART II

System Administrator's Guide

Installing, configuring, securing, deploying and maintaining the application — including the authentication model, the reverse-proxy requirements it depends on, and the runbook for when something breaks.

Who this part is for

Whoever operates the server. Assumes a working knowledge of the command line, Node.js and nginx. Read sections 12 to 14 before exposing the dashboard to a network.

8 Architecture

Layer	Technology
Framework	Next.js 16 (App Router, Turbopack)
Language	TypeScript 5, target ES2017
UI	React 19, Tailwind CSS 3, Radix primitives
Images	Next.js Image, Sharp for upload re-encoding
Contact form	EmailJS, entirely client-side
Content store	JSON and BibTeX files on disk — no database
Auth	Single shared password, HMAC-SHA256 signed cookie, Web Crypto

8.1 Directories that matter

content/	Editable content, written by the dashboard
├ members.json	Faculty, postdocs, scholars, staff, alumni
├ projects.json	Ongoing projects, completed-projects PDF, sponsors
├ research.json	Research areas and their topics
├ gallery.json	Albums and categories
├ facilities.json	Instruments, in-charges, stats, slot charges
└ .backups/	Rolling snapshots before each save (gitignored)
public/	
├ data/publications.bib	~180 BibTeX entries
├ data/metrics.json	Citations, h-index, source, lastUpdated
└ images/	Uploads land here, by destination folder
src/	
├ proxy.ts	Auth + network gate on /admin and /api/admin
├ lib/admin-auth.ts	Session tokens, password check, cookie options
├ lib/admin-network.ts	X-Forwarded-For parsing, CIDR allowlist, origin
├ lib/content.ts	Atomic reads/writes, backups, revalidation
├ lib/content-schema.ts	Validation and normalisation of every write
└ components/admin/	Dashboard shell, fields, session timeout

ONE GATE, NO PER-ROUTE CHECKS

`src/proxy.ts` runs ahead of every `/admin` page and `/api/admin` endpoint. Address allowlist, `Origin` check and session validation all live there, which is why no individual route can ship without them. Read it first when auditing.

9 Installation and the build

9.1 Requirements

- **Node.js 20.9 or newer** — required by Next.js 16.
- **npm**, or pnpm (a `pnpm-lock.yaml` is committed).
- A build host able to compile Sharp, or one of its prebuilt binaries.

9.2 First run

```
git clone https://github.com/Ankush4github/CBRL-Website-v2-With-CMS.git
cd CBRL-Website-v2-With-CMS
npm install

cp .env.example .env.local      # then edit it - see section 10
npm run dev                    # http://localhost:3000
```

9.3 Production

```
npm run build
npm start                # serves the build, default port 3000
```

9.4 Scripts

Command	Purpose
<code>npm run dev</code>	Development server with Turbopack.
<code>npm run build</code>	Production build. Fails on type errors.
<code>npm start</code>	Serve the production build.
<code>npm run lint</code>	ESLint over <code>src</code> .
<code>npm run typecheck</code>	TypeScript, no emit.
<code>npm run optimize-images</code>	Batch-compress <code>public/images</code> with Sharp.
<code>npm run test:network</code>	Exercises IP parsing and CIDR matching — the logic the throttle and allowlist depend on. Run after changing proxy settings.

10 Environment variables

All configuration lives in `.env.local`, which is gitignored. Restart the server after any change. `.env.example` is the annotated template.

10.1 Required for the dashboard

Variable	Purpose
<code>ADMIN_PASSWORD</code>	The single shared dashboard password. Make it long — it is the entire perimeter. Compared by SHA-256 digest, so its length never leaks through timing.
<code>ADMIN_SESSION_SECRET</code>	Signs the session cookie. Minimum 16 characters ; the app refuses to issue sessions below that. Changing it signs everybody out immediately.

Generate a secret with:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

IF THESE ARE UNSET

The dashboard refuses every sign-in and shows a setup notice instead of a login form. This is a deliberate fail-closed default: an unconfigured deployment is locked, never open.

10.2 Session timing

Variable	Default	Purpose
<code>ADMIN_IDLE_TIMEOUT_MINUTES</code>	5	Minutes of inactivity before a session ends. Accepts fractions (<code>0.5</code>). Anything unparseable or ≤ 0 falls back to 5 rather than to "no limit"; values above the 8-hour absolute cap are clamped to it.

The 8-hour absolute cap is a compile-time constant (`SESSION_MAX_AGE_SECONDS` in `src/lib/admin-auth.ts`) and is not configurable by environment.

RAISING THE IDLE TIMEOUT

Five minutes is short by design, but the editors hold unsaved drafts in the browser. If your editors write long-form content and complain about losing work, raising this to 15 or 30 minutes is reasonable — a timeout people work around is worse than a longer one they respect.

10.3 Network and proxy

Variable	Default	Purpose
<code>TRUSTED_PROXY_HOPS</code>	1	How many reverse proxies sit in front of the app. <code>1</code> for a single nginx; <code>0</code> if Node is exposed directly, which makes <code>X-Forwarded-For</code> ignored entirely. Getting this wrong silently disables the login throttle — see section 14.
<code>ADMIN_ALLOWED_IPS</code>	unset	Optional. Comma-separated IPs and CIDR ranges, IPv4 and IPv6, e.g. <code>10.111.0.0/16, 203.0.113.5</code> . Restricts <code>/admin</code> and <code>/api/admin</code> only; the public site is untouched. Unset means password-only.
<code>ADMIN_ORIGIN</code>	unset	Optional. Pins the origin that dashboard writes must come from, e.g. <code>https://cbrl.iitkgp.ac.in</code> . Leave unset to derive it from <code>X-Forwarded-Proto</code> and <code>X-Forwarded-Host</code> .

10.4 Contact form

Variable	Purpose
<code>NEXT_PUBLIC_EMAILJS_SERVICE_ID</code>	EmailJS credentials for the contact form. Without them the form renders but submission fails. <code>NEXT_PUBLIC_</code> means these are visible in the browser bundle — that is inherent to EmailJS, so use a key scoped to sending only.
<code>NEXT_PUBLIC_EMAILJS_TEMPLATE_ID</code>	
<code>NEXT_PUBLIC_EMAILJS_PUBLIC_KEY</code>	

11 How content is stored and revalidated

There is no database. Pages read JSON from disk on the server, so a dashboard save writes a file and the change is live on the next request — no rebuild, no redeploy.

11.1 The write path

- 1 **Validate.** The whole document is checked and normalised. A single bad field rejects the entire save with a 422 and a per-field issue list; nothing is written.
- 2 **Back up.** The current file is copied to `content/.backups/<collection>/<timestamp>.json`. The most recent **20** are kept per collection; older ones are pruned automatically.
- 3 **Write atomically.** Content goes to a temporary file in the same directory and is then renamed over the target. A crash or a concurrent read can never observe a half-written file.
- 4 **Revalidate.** `revalidatePath()` drops the cached render of the pages that depend on the collection.

11.2 Which pages each collection revalidates

Collection	Paths revalidated
<code>members</code>	<code>/members</code> , <code>/facilities</code>
<code>projects</code>	<code>/projects</code>
<code>research</code>	<code>/research</code>
<code>gallery</code>	<code>/gallery</code>
<code>facilities</code>	<code>/facilities</code>
<code>metrics</code>	<code>/publications</code> , <code>/about-the-pi</code>
<code>publications.bib</code>	<code>/publications</code> , <code>/about-the-pi</code> , <code>/</code>

11.3 Uploads

Uploaded bytes are re-encoded through Sharp to WebP at quality 82 and given a content-hashed filename. **Re-encoding is the validation** — the caller's `Content-Type` and filename are never trusted, and anything Sharp cannot decode is refused, so no caller-chosen file can be served back from this origin.

Destination	Folder	Max width	Sub-folders
<code>members</code>	<code>public/images/MembersDP</code>	900 px	No
<code>alumni</code>	<code>public/images/AlumniDP</code>	900 px	No
<code>facilities</code>	<code>public/images/facilities</code>	1600 px	No
<code>gallery</code>	<code>public/images/gallery</code>	2000 px	Yes
<code>sponsors</code>	<code>public/images/sponsors</code>	800 px	No

Maximum accepted size is 12 MB, checked against `Content-Length` before the body is read into memory.
Editing the JSON and BibTeX files by hand still works exactly as before — the dashboard is another way in, not a replacement.

12 Authentication and session model

12.1 The session cookie

One shared password buys an HMAC-SHA256 signed cookie. Signing uses Web Crypto rather than `node:crypto`, so identical code verifies the cookie in the Edge-runtime proxy and inside Node route handlers.

Property	Value
Name (production)	<code>__Host-cbrl_admin_session</code>
Name (development)	<code>cbrl_admin_session</code>
Flags	<code>httpOnly</code> , <code>SameSite=Lax</code> , <code>Secure</code> in production, <code>Path=</code>
<code>Max-Age</code>	The idle timeout, re-issued on every authenticated request

The `__Host-` prefix binds the cookie to the exact host: the browser accepts it only with `Secure`, `Path=` and no `Domain`, so no sibling `*.iitkgp.ac.in` site can set or shadow it. The prefix is dropped in development because a `__Host-` cookie without `Secure` is refused outright and `npm run dev` serves plain HTTP.

The cookie carries no secret of its own — only two timestamps and a signature over them. A stolen cookie therefore expires on its own, and rotating `ADMIN_SESSION_SECRET` invalidates every session at once.

12.2 Absolute expiry and idle timeout

The signed payload holds two clocks, and the session dies to whichever fires first:

Field	Meaning	Behaviour
<code>exp</code>	Absolute expiry	Fixed at sign-in, 8 hours out, and never moved . No amount of activity extends a session beyond it.
<code>seen</code>	Last activity	Slid forward by the proxy on every authenticated request. Once it falls further behind than the idle timeout, the session is over.

Keeping `seen` inside the signed payload is what makes the idle timeout real rather than decorative: the browser cannot move it, so a client that never runs the in-page timer is still signed out by the server. The `Max-Age` on the cookie is set to the idle timeout as a second line of defence — a browser closed for longer than that has nothing left to send.

The proxy distinguishes an expired-by-idle session from an absent or invalid one. Page requests are redirected to `/admin/login?reason=timeout` (preserving the page they were on via `next=`), and API requests receive `401` with `{"reason": "timeout"}`. Both responses clear the cookie.

12.3 The activity heartbeat

The editors keep drafts in the browser and only contact the server on save, so somebody typing a long abstract generates no requests for minutes — indistinguishable, to a request-counting timeout, from an empty chair. `src/components/admin/SessionTimeout.tsx` closes that gap:

- Pointer, keyboard, wheel, scroll and touch events count as activity, throttled to once a second.

- When activity has occurred, a `POST /api/admin/heartbeat` slides the server-side clock, at most about once every quarter of the idle timeout.
- A warning panel appears 60 seconds before expiry; reaching zero signs the user out.
- Activity is shared between tabs through `localStorage`, so work in one tab keeps the others alive.
- Everything is measured against wall-clock time rather than counted down, so a slept laptop or a throttled background tab still reaches the right conclusion on its next tick.

THE HEARTBEAT CANNOT KEEP AN IDLE SESSION ALIVE

It fires only when genuine interaction has been recorded since the previous one. An untouched tab sends nothing and times out normally. It is also a `POST`, so the `Origin` check applies and no other site can trigger it.

13 Security layers

The password is the perimeter, so the layers around it carry the weight.

Layer	What it does
Session cookie	HMAC-SHA256 signed, <code>httpOnly</code> , <code>Secure</code> , 8-hour absolute cap and 5-minute idle timeout. Host-bound by the <code>__Host-</code> prefix.
Login throttle	5 attempts per address, then a 10-minute block. The address is read from the <i>right-hand</i> end of <code>X-Forwarded-For</code> — reading from the left would let a caller mint a fresh identity per request and guess indefinitely. An unidentifiable address falls into one shared bucket, so a misconfigured proxy throttles everyone together rather than nobody.
Origin check	Writes to <code>/api/admin/*</code> must carry an <code>Origin</code> matching the site. <code>SameSite=Lax</code> alone is insufficient here because it treats every <code>*.iitkgp.ac.in</code> host as same-site, so a compromised neighbour could otherwise drive authenticated writes.
IP allowlist	Optional <code>ADMIN_ALLOWED_IPS</code> , applied ahead of everything else including the login endpoint. An allowlist consisting only of unparseable entries is treated as unset, so a typo cannot lock everybody out permanently.
Upload hardening	Re-encoded through Sharp to <code>.webp</code> ; the declared type and filename are never trusted. Destination folders are chosen from a fixed set, so no request can write outside <code>public/images</code> .
Write safety	Validated, then temp-file-and-rename, with the previous 20 versions retained.
Path validation	Saved asset paths must begin with a single <code>/</code> and may not contain <code>..</code> , so no record can point the site at another origin or escape <code>public/</code> .
Response headers	A strict CSP, HSTS, <code>X-Frame-Options: DENY</code> , <code>nosniff</code> and a restrictive <code>Permissions-Policy</code> are set site-wide in <code>next.config.js</code> . Everything under <code>/admin</code> is additionally <code>no-store</code> .

SCRIPTING AGAINST THE API

The `Origin` check rejects non-browser clients such as `curl`, which never send that header. Pass `-H "Origin: https://cbrl.iitkgp.ac.in"` when scripting writes.

14 Reverse proxy configuration

The login throttle and the IP allowlist both depend on nginx replacing or appending to `X-Forwarded-For` rather than passing the caller's value through untouched. Next.js 16 exposes no socket address in either runtime, so this header is the only source of a client address.

```
location / {
    proxy_pass          http://127.0.0.1:3000;
    proxy_http_version 1.1;

    proxy_set_header    Host          $host;
    proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header    X-Forwarded-Proto $scheme;
    proxy_set_header    X-Forwarded-Host $host;

    proxy_set_header    Upgrade      $http_upgrade;
    proxy_set_header    Connection  "upgrade";
}
```

`$proxy_add_x_forwarded_for` is the important one: it *appends* the peer nginx actually spoke to, which is what makes the right-hand end of the list trustworthy.

14.1 Verifying it end to end

Do this after every proxy change, and **before** setting `ADMIN_ALLOWED_IPS`, so you allowlist the address the server really sees. Sign in, then:

```
curl -H "Cookie: <session cookie>" \
-H "X-Forwarded-For: 1.2.3.4" \
https://cbrl.iitkgp.ac.in/api/admin/whoami
```

The `clientIp` in the response must be **your real address**. If it comes back as `1.2.3.4`, nginx is forwarding the caller's header unchanged, the throttle is protecting nothing, and an allowlist would be trivially bypassed. Fix the proxy configuration before relying on either.

The same endpoint reports `trustedProxyHops`, whether the allowlist is on, the expected origin, and the raw forwarded headers — enough to diagnose a mismatch from one response.

SETTING TRUSTED_PROXY_HOPS WRONGLY

Too high and the app reads an entry the caller supplied, so every request appears to come from a different address and the throttle never fires. Too low and it reads nginx's own address, so all users share one bucket and five failures anywhere lock out everybody. Confirm with `/api/admin/whoami` rather than assuming.

15 Deployment and keeping git in step

15.1 Deploying an update

```
git pull
npm ci
npm run build
# restart the process manager, e.g.
sudo systemctl restart cbrl-website
```

PRODUCTION EDITS LIVE ON THE SERVER'S DISK

Everything the dashboard writes — `content/*.json`, `public/data/publications.bib`, `public/data/metrics.json` and new files under `public/images/` — exists only on the server until somebody copies it back. **A redeploy from a stale checkout will overwrite months of editorial work.**

15.2 Pulling editorial changes back into git

Do this regularly, and always immediately before a deployment:

```
# on your workstation, from the repository root
scp -r user@server:/srv/cbrl/content/*.json      ./content/
scp      user@server:/srv/cbrl/public/data/publications.bib ./public/data/
scp      user@server:/srv/cbrl/public/data/metrics.json    ./public/data/
rsync -av user@server:/srv/cbrl/public/images/          ./public/images/

git add content public/data public/images
git commit -m "Sync editorial content from production"
git push
```

A weekly cron job doing the same thing and committing on a branch removes the risk of forgetting.

16 Deploying elsewhere (Vercel and other hosts)

The application assumes a **writable, persistent filesystem**. That assumption holds on a conventional server and fails on most serverless platforms.

THE DASHBOARD CANNOT SAVE ON VERCEL

Vercel's filesystem is read-only outside `/tmp`, and instances are ephemeral and not shared. Content saves and image uploads both write into the project directory at runtime, so on Vercel every save returns *"Could not write the file on the server"* and every upload returns *"Could not write the image on the server"*. The public site is unaffected — its pages are generated at build time from the committed content/`*.json`.

Host type	Public site	Dashboard
VPS / bare metal / campus server	Works	Works
Docker with a persistent volume	Works	Works — mount content/ and public/images/
Vercel, Netlify Functions, Cloudflare Workers	Works	Saves and uploads fail

Two secondary effects are worth knowing on any serverless host. The login throttle keeps its counters in memory, so separate instances do not share them and the effective attempt limit is higher than five. And `revalidatePath()` only affects the instance that handled the request.

Making the dashboard work on such a platform means replacing the storage layer — moving `content/` to a database or object store such as Vercel Blob, and uploads to the same, which touches `src/lib/content.ts`, the upload route and the publications and metrics writers. If you deploy to Vercel as a preview, leave `ADMIN_PASSWORD` and `ADMIN_SESSION_SECRET` unset so the dashboard is locked rather than half-working.

17 API reference

Every endpoint sits behind the proxy gate: address allowlist, then `Origin` on writes, then a valid non-idle session. `/api/admin/login` and `/api/admin/logout` are reachable without a session; everything else is not.

Method	Path	Purpose
POST	<code>/api/admin/login</code>	Exchange the password for a session. Throttled. <code>401</code> wrong, <code>429</code> blocked, <code>503</code> unconfigured.
POST	<code>/api/admin/logout</code>	Clear the session cookie.
POST	<code>/api/admin/heartbeat</code>	Report activity; slides the idle clock. Returns the idle timeout in seconds.
GET	<code>/api/admin/whoami</code>	Diagnostics: client IP, allowlist state, trusted hops, expected origin, raw forwarded headers.
GET	<code>/api/admin/content/[collection]</code>	Read one collection.
PUT	<code>/api/admin/content/[collection]</code>	Validate and write one collection. <code>422</code> with a per-field issue list on failure.
GET	<code>/api/admin/publications</code>	List BibTeX entries with duplicate-key flags.
POST	<code>/api/admin/publications</code>	Add an entry, from a form payload or pasted BibTeX.
PUT / DELETE	<code>/api/admin/publications/[id]</code>	Edit or remove one entry, leaving the rest byte-identical.
POST	<code>/api/admin/publications/doi</code>	Fetch BibTeX from doi.org by DOI. 12-second timeout.
GET / PUT	<code>/api/admin/metrics</code>	Read or write citation metrics.
POST	<code>/api/admin/upload</code>	Upload an image. Multipart, 12 MB cap, re-encoded to WebP.

18 Backup and restore

18.1 What the application keeps for you

Before every save the previous version of the file is copied to `content/.backups/<collection>/<ISO-timestamp>.json`, and the most recent 20 per collection are retained. Publications are kept the same way as `.bib` files. This is an "undo the last few edits" facility, **not** a backup strategy — it lives on the same disk and is excluded from git.

18.2 Restoring a previous version

```
ls -lt content/.backups/members/          # newest first
cp content/.backups/members/2026-07-27T09-14-22-431Z.json content/members.json
```

Changes appear on the next request; no restart is needed. Copy rather than move, so the backup survives.

18.3 A real backup

Back up nightly, off the machine, covering:

- `content/` — including `.backups/`
- `public/data/` — publications and metrics
- `public/images/` — every upload ever made
- `.env.local` — store this with your secrets, not alongside the content

```
tar czf cbrl-$(date +%F).tar.gz \
    content public/data public/images
```

19 Routine maintenance

Task	Frequency	Notes
Sync content into git	Weekly	Section 15.2. Mandatory before any redeploy.
Verify backups restore	Quarterly	A backup never test-restored is a guess.
Rotate <code>ADMIN_PASSWORD</code>	On personnel change	Anyone who has ever had it retains access until it changes.
Rotate <code>ADMIN_SESSION_SECRET</code>	Yearly, or on suspicion	Signs everyone out immediately. Safe to do at any time.
Re-verify <code>/api/admin/whoami</code>	After proxy changes	Section 14.1.
<code>npm audit</code> / dependency updates	Quarterly	Run <code>npm run build</code> , <code>lint</code> and <code>typecheck</code> before deploying.
Prune <code>public/images/</code>	Yearly	Replaced uploads are never deleted automatically. Cross-check against the JSON before removing anything.

20 Administrator troubleshooting

Symptom	Cause and remedy
"The dashboard is not configured"	<code>ADMIN_PASSWORD</code> is missing, or <code>ADMIN_SESSION_SECRET</code> is shorter than 16 characters. Set both in <code>.env.local</code> and restart.
Editors report "Could not write the file on the server"	The process cannot write to <code>content/</code> or <code>public/</code> . Check ownership and permissions, and free disk space with <code>df -h</code> . On a serverless host this is expected — section 16.
Everyone is locked out by the throttle	<code>TRUSTED_PROXY_HOPS</code> is too low, so all users share one bucket. Verify with <code>/api/admin/whoami</code> . Restarting the process clears the in-memory counters.
The throttle never blocks anyone	<code>TRUSTED_PROXY_HOPS</code> is too high, or nginx is passing the caller's <code>X-Forwarded-For</code> through. Section 14 — treat as urgent, since the password is then exposed to unlimited guessing.
Sessions end far sooner than expected	Check <code>ADMIN_IDLE_TIMEOUT_MINUTES</code> . Also confirm server time is correct — a clock that jumps forward expires sessions early. <code>timedatectl</code> .
Everyone signed out at once	<code>ADMIN_SESSION_SECRET</code> changed, or the deployment rolled back to a build with a different one. Expected behaviour; sign in again.
"This request did not come from the dashboard" for everyone	The origin check is failing. nginx is probably not sending <code>X-Forwarded-Proto</code> or <code>X-Forwarded-Host</code> . Either fix the proxy or pin <code>ADMIN_ORIGIN</code> .
Legitimate users get "not available from your network"	<code>ADMIN_ALLOWED_IPS</code> excludes them, or the allowlist is on while the address cannot be established — which is refused by design. The message names the address the server saw; add it, or unset the variable to fall back to password-only.
Uploads fail with "could not be read as an image"	Sharp cannot decode the file. If every upload fails, Sharp is likely not installed correctly for this platform — reinstall with <code>npm ci</code> on the target architecture.
A saved change is not on the public page	Confirm the file on disk actually changed. If it did, the cached render was not dropped — restart the process to clear it, then check for errors from <code>revalidatePath</code> in the logs.
Content reverted after a deployment	A stale checkout overwrote production edits. Restore from <code>content/.backups/</code> or your off-machine backup, then adopt the sync routine in section 15.2.

FIRST DIAGNOSTIC TO REACH FOR

`GET /api/admin/whoami`, signed in, answers most networking questions in one response: the address the server attributes to the caller, whether the allowlist is active, the trusted-hop count, the expected origin, and the raw forwarded headers behind all of it.